



MILESTONE 1

OLTP Database

Building and seeding the normalized Azure SQL storefront database



STEP 1

Create Azure SQL Database (Free Offer)

WHAT

Created a database named `atliq_commerce` using the SQL database Free Offer (serverless, General Purpose) inside resource group `rg-atliq-capstone`, region Central India, with SQL authentication and admin login `atliq_admin`.

WHY

The Free Offer provides 100,000 vCore seconds and 32 GB per month at no cost for the life of the subscription. Setting the free-limit behaviour to auto-pause guarantees the OLTP layer can never incur a charge.

Create Azure SQL Database (Free Offer)

The screenshot displays the Microsoft Azure portal interface. At the top, the header bar shows the Microsoft Azure logo, a search bar, and the user profile 'ashish@alphainterbiz.co...'. The main content area is titled 'Home' and shows the 'atliq_commerce (atliq-sql-ab/atliq_commerce)' SQL database. The left sidebar contains a navigation menu with options like Overview, Activity log, Tags, Diagnose and solve problems, Query editor, Mirror database in Fabric (preview), Resource visualizer, Settings, Data management, Integrations, Power Platform, Security, Intelligent performance, Monitoring, Automation, and Help. The main content area is divided into two sections: 'Essentials' and 'Start working with your database'. The 'Essentials' section displays key properties of the database, including Resource group, Status, Location, Subscription, and Tags. The 'Start working with your database' section provides four actionable cards: 'Configure access', 'Connect to application', 'Start developing', and 'Mirror database in Fabric'. Each card includes a brief description and a 'Learn more' link.

atliq_commerce (atliq-sql-ab/atliq_commerce) SQL database

Search resources, services, and docs (G+/)

Copilot

ashish@alphainterbiz.co... ALPHA INTERBIZ LLP (ALPHAINT...)

Home

atliq_commerce (atliq-sql-ab/atliq_commerce)

SQL database

List the access control settings for this SQL database. Summarize the properties for this SQL database. Diagnose login failure errors

Search

Copy Restore Export Set server firewall Delete Open in Feedback

Overview

- Activity log
- Tags
- Diagnose and solve problems
- Query editor
- Mirror database in Fabric (preview)
- Resource visualizer
- Settings
- Data management
 - Replicas
 - Sync to other databases
- Integrations
 - Azure Synapse Link
 - Stream analytics (preview)
 - Add Azure AI Search
- Power Platform
 - Power BI
 - Power Apps
 - Power Automate
- Security
- Intelligent performance
- Monitoring
- Automation
- Help

Essentials

Resource group (move) : rg-atliq-capstone

Status : Online

Location : Central India

Subscription (move) : Azure subscription 1

Subscription ID : fe7a04e5-37b1-4992-a127-541400ea45ad

Tags (edit) : Add tags

Server name : atliq-sql-ab.database.windows.net

Connection strings : Show database connection strings

Pricing tier : Free - General Purpose - Serverless: Gen5, 2 vCores

Overage billing : Disabled

Free monthly vCore amount : 92,053 vCore seconds remaining

Earliest restore point : 2026-09-22 11:34 UTC

JSON View

Getting started Monitoring Properties Features Notifications (0) Integrations Tutorials

Start working with your database

Connect to your database and start working with data with a few simple steps. [Learn more](#)

Configure access

Configure network access to your SQL server. [Learn more](#)

Connect to application

Use connection strings to connect to your SQL database from your applications and favorite tools.

Start developing

Work in your database by using tools to add, modify and query data. [Compare tools](#)

Mirror database in Fabric

Replicate existing databases in Fabric, and help your team achieve streamlined ETL and operational analytics goals. [Learn more](#)

Configure

See connection strings

Open in Visual Studio Code

Open in SQL Server Management Studio

Add or remove favorites by pressing Ctrl+Shift+F

atliq_commerce Overview: Free, General Purpose, Serverless Gen5, Central India, rg-atliq-capstone, status Online



STEP 2

Configure Networking and Firewall

WHAT

Set public network access to Selected networks, added my current client IP as a firewall rule, and enabled Allow Azure services and resources to access this server.

WHY

The client IP rule lets me connect from VS Code. Allowing Azure services is required so Azure Data Factory can reach the database during the ingestion milestone (M2) without a separate firewall change.

Configure Networking and Firewall

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

ashish@alphainterbiz.co...
ALPHA INTERBIZ LLP (ALPHAINT...)

Home > atliq_commerce (atliq-sql-ab/atliq_commerce) > atliq-sql-ab

atliq-sql-ab | Networking ☆ ...
SQL server

Search

Feedback

Overview
Activity log
Access control (IAM)
Tags
Quick start
Diagnose and solve problems
Resource visualizer
Settings
Data management
Security

Networking

Microsoft Defender for Cloud
Transparent data encryption
Identity
Auditing
Intelligent performance
Monitoring
Automation
Help

Public access Private access Connectivity

Public network access
Public Endpoints allow access to this resource through the internet using a public IP address. An application or resource that is granted access with the following network rules still requires proper authorization to access this resource. [Learn more](#)

Public network access

☐ Disable
☒ Selected networks

Connections from the IP addresses configured in the Firewall rules section below will have access to this database. By default, no public IP addresses are allowed. [Learn more](#)

Virtual networks
Allow virtual networks to connect to your resource using service endpoints. [Learn more](#)

+ Add a virtual network rule

Rule	Virtual network	Subnet	Address range	Endpoint status	Resource group	Subscription	State
------	-----------------	--------	---------------	-----------------	----------------	--------------	-------

Firewall rules
Allow certain public internet IP addresses to access your resource. [Learn more](#)

+ Add your client IPv4 address () + Add a firewall rule

Rule name	Start IPv4 address	End IPv4 address	
home			
QueryEditorClientIPAddress_17898490...			

Exceptions

☒ Allow Azure services and resources to access this server ⓘ

Save Discard

Add or remove favorites by pressing Ctrl+Shift+F

atliq-sql-ab Networking: Selected networks, client IP firewall rules (IPs blurred), Allow Azure services ticked



STEP 3

Connect from VS Code

WHAT

Connected to the database from VS Code using the MSSQL extension, SQL Login authentication, user atliq_admin, with the target database explicitly set to atliq_commerce.

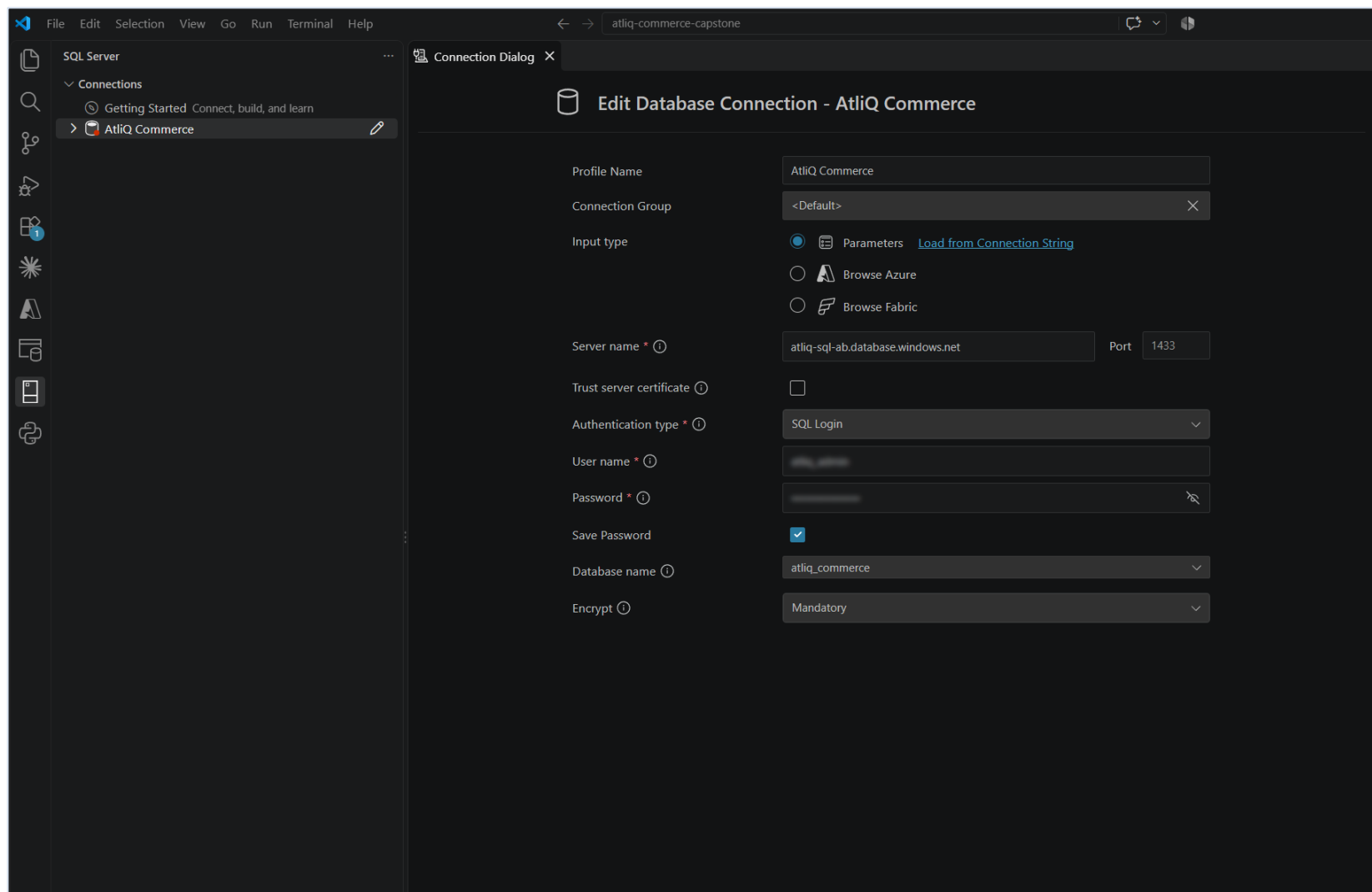
WHY

SQL Login with the admin account is the reliable path since Entra login was not configured. Explicitly selecting the database avoids accidentally running scripts against the master system database, a mistake that fails all table creation.

3

STEP 3 • SCREENSHOT

Connect from VS Code



VS Code MSSQL connection: atliq-sql-ab.database.windows.net, SQL Login, database atliq_commerce (credentials blurred)



STEP 4

Create the OLTP Schema

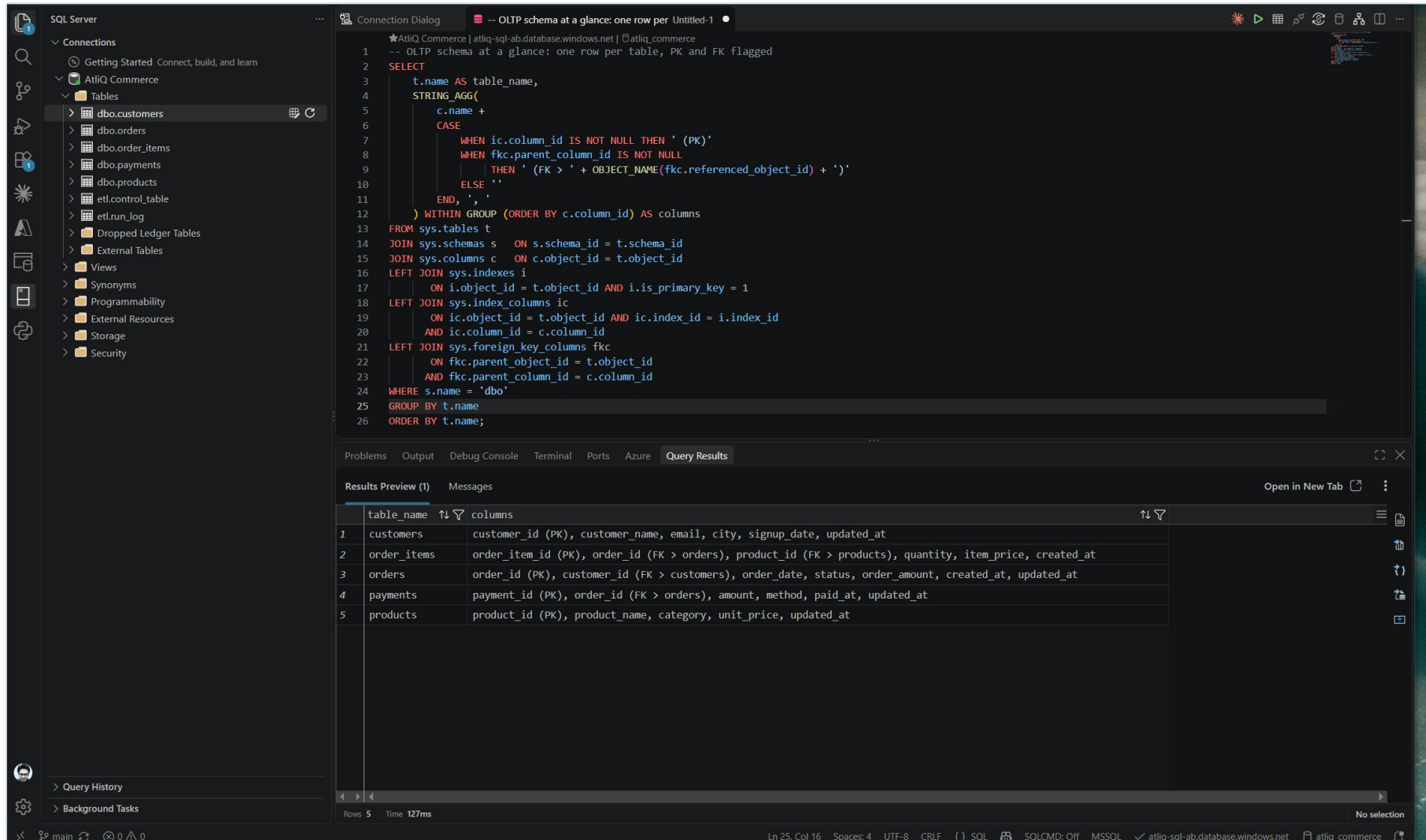
WHAT

Ran 01_schema_ddl.sql to create five normalized (3NF) tables: customers, products, orders, order_items, payments, each with primary keys, foreign keys, and an updated_at or created_at timestamp column.

WHY

A normalized schema mirrors a real transactional storefront (fast writes, no duplication). The timestamp columns become the watermarks that the incremental nightly sync relies on in later milestones.

Create the OLTP Schema



The screenshot shows the Visual Studio Code interface with a SQL query in the main editor and its results in the Results Preview pane.

SQL Query:

```
1  -- OLTP schema at a glance: one row per table, PK and FK flagged
2  SELECT
3      t.name AS table_name,
4      STRING_AGG(
5          c.name +
6          CASE
7              WHEN ic.column_id IS NOT NULL THEN ' (PK)'
8              WHEN fkc.parent_column_id IS NOT NULL
9                  THEN ' (FK > ' + OBJECT_NAME(fkc.referenced_object_id) + ')'
10             ELSE ''
11          END, ','
12      ) WITHIN GROUP (ORDER BY c.column_id) AS columns
13 FROM sys.tables t
14 JOIN sys.schemas s ON s.schema_id = t.schema_id
15 JOIN sys.columns c ON c.object_id = t.object_id
16 LEFT JOIN sys.indexes i
17     ON i.object_id = t.object_id AND i.is_primary_key = 1
18 LEFT JOIN sys.index_columns ic
19     ON ic.object_id = t.object_id AND ic.index_id = i.index_id
20     AND ic.column_id = c.column_id
21 LEFT JOIN sys.foreign_key_columns fkc
22     ON fkc.parent_object_id = t.object_id
23     AND fkc.parent_column_id = c.column_id
24 WHERE s.name = 'dbo'
25 GROUP BY t.name
26 ORDER BY t.name;
```

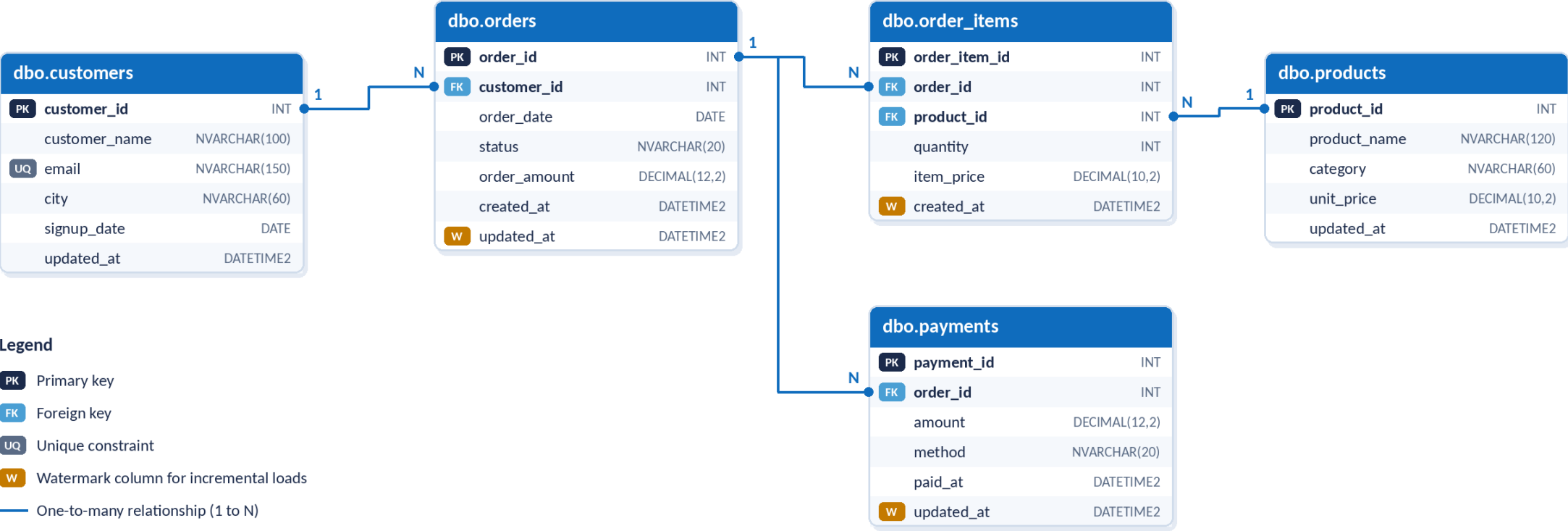
Results Preview (1):

table_name	columns
customers	customer_id (PK), customer_name, email, city, signup_date, updated_at
order_items	order_item_id (PK), order_id (FK > orders), product_id (FK > products), quantity, item_price, created_at
orders	order_id (PK), customer_id (FK > customers), order_date, status, order_amount, created_at, updated_at
payments	payment_id (PK), order_id (FK > orders), amount, method, paid_at, updated_at
products	product_id (PK), product_name, category, unit_price, updated_at

Rows: 5 | Time: 127ms

Schema query in VS Code: the five dbo tables, one row each, with PK and FK columns flagged

Create the OLTP Schema



Checks: unit_price >= 0, quantity > 0

Source: 01_schema_ddl.sql | atliq_commerce (Azure SQL)

Schema diagram drawn from 01_schema_ddl.sql: five 3NF tables, their keys, and the watermark columns



STEP 5

Load the Seed Data

WHAT

Ran the insert scripts 02 to 06 in strict order to load 40 customers, 25 products, 300 orders, 783 order_items, and 246 payments.

WHY

Order matters because of foreign keys: a line item cannot reference an order that does not yet exist. Parent tables are loaded before child tables so every foreign key resolves cleanly.



STEP 6

Create the ETL Control Table

WHAT

Ran 07_etl_control_table.sql to create etl.control_table (one row per source, each with a load_type and watermark_column) and the usp_update_watermark stored procedure.

WHY

This table is the metadata that drives the whole ingestion pipeline. One generic pipeline reads it and decides how to load each source, so a new source is added by inserting a row rather than by editing any pipeline.



STEP 7

Create the Audit Run Log

WHAT

Ran 08_audit_run_log.sql to create etl.run_log and the log-start and log-finish stored procedures under a dedicated etl schema.

WHY

This is the pipeline's black box recorder for the reliability milestone (M7). Each run records what ran, when, how many rows, and pass or fail. It stays dormant until the pipeline calls it in later milestones.



STEP 8

Verify and Reconcile

WHAT

Ran `verify_counts.sql` to confirm exact row counts (40 / 25 / 300 / 783 / 246) and a revenue reconciliation where the order header total equals the sum of the line items.

WHY

Verifying counts and reconciling revenue three independent ways proves the seed data loaded completely and is internally consistent before any downstream transformation work begins. This is the M1 deliverable.

Verify and Reconcile

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Connections' pane shows 'AtliQ Commerce' expanded, with 'Tables' listed. The 'Query History' pane shows 'Background Tasks' with 'No background tasks'. The main query window displays a script named 'verify_counts.sql' with the following content:

```
1  -- =====
2  -- AtliQ Commerce | Load verification (run after M1 seed load)
3  -- Expected: 40 customers, 25 products, 300 orders, 783 order_items, 246 payments
4  -- =====
5  SELECT 'customers' AS tbl, COUNT(*) AS [rows] FROM dbo.customers
6  UNION ALL SELECT 'products', COUNT(*) FROM dbo.products
7  UNION ALL SELECT 'orders', COUNT(*) FROM dbo.orders
8  UNION ALL SELECT 'order_items', COUNT(*) FROM dbo.order_items
9  UNION ALL SELECT 'payments', COUNT(*) FROM dbo.payments;
10
11 -- Revenue reconciliation: three independent sums must match.
12 -- (order_amount on the header) = (sum of its line items) = (payment amount)
13 SELECT
14     (SELECT SUM(order_amount) FROM dbo.orders) AS header_total,
15     (SELECT SUM(quantity * item_price) FROM dbo.order_items) AS line_item_total,
16     (SELECT SUM(amount) FROM dbo.payments) AS payment_total;
17
```

The 'Query Results' pane at the bottom shows the results of the query. It contains two tables. The first table, 'tbl', has two columns: 'tbl' and 'rows'. The second table, 'header_total', has three columns: 'header_total', 'line_item_total', and 'payment_total'.

tbl	rows
customers	40
products	25
orders	300
order_items	783
payments	246

header_total	line_item_total	payment_total
2076041.00	2076041.00	1745772.00

verify_counts.sql: 40 / 25 / 300 / 783 / 246, and header_total = line_item_total = 2,076,041.00

Milestone 1 Complete

What this milestone delivered

OLTP database

Five 3NF tables created, seeded, and verified in Azure SQL. The operational source of the whole pipeline.

Control table

etl.control_table seeded with 5 sources and their load rules. Ready to drive the M2 ingestion pipeline.

Audit log

etl.run_log and its stored procedures created. Ready to record pipeline runs for the M7 reliability work.

Verified load

40

customers

25

products

300

orders

783

order items

246

payments

Revenue reconciled at 2,076,041.00 (order headers = line items). Cost to date: Free (SQL Free Offer, auto-pause).